weirdspacingkeymap.c

5 layers · 34 keys · 2 combos

every arrow starts on the key that triggers it

how to read

- pink key = press it to go to another layer
- violet key = press it to toggle a layer on and off
- red key(s) = held down while you are on this layer
- solid arrow = hold
- dashed arrow = toggle / go to / one-shot
- combo combo = press the red keys together

0 layer 0

base layer

TD Q Esc	W	E	R	T	Y	U	I	O	P
A	S Alt	D Ctrl	F Shift	G	H	J RShift	K RCtrl	L Alt	;
Z GUI	X	C	V	B	N	M	,	.	/ RGUI
			Tab Llck layer 1	Space layer 2	Bksp Llck layer 3	Enter layer 4			

4 layer 4

	Py Crl	Py Brk	Py Par			Nm Thz	Em Fyou	Ctrl+Alt+Del	
			Llck			Llck			Bt Grb

3 layer 3

Shift	Home	Ins	Del	End	←	→		Uc Next	🔊
Ctrl	Ms Whll	Ms Whlu	Ms Whlr	PgUp	↑	Move ←	Move ↑	Ms Rght	🔊
Alt	Btn 1	Ms Whld	Btn 2	PgDn	↓	Btn 1	Move ↓	Btn 2	🔇
			▽ Llck ▽			Llck ▽			

2 layer 2

F4	F3	F2	←	→	=	7	8	9	*
F8	PrtSc	Caps	Enter	↑	.	4 RShift	5 RCtrl	6 Alt	KP -
F12	F11	F10	BOOT	↓	0	1	2	3	+
			▽ Llck			▽ Llck ▽			

1 layer 1

`	~	!	@	#	$	%	^	&	*
_	{	[	(	UC(0x2191)	UC(0x2192)	)	]	}	"
UC(0x00F7)	=	KP -	+	UC(0x2190)	UC(0x2193)	\	|	'	UC(0x00B0)
			Llck ▽			▽ Llck ▽			

hold

hold

hold

hold

weirdspacingkeymap.c

keymap source, original code (comments kept)

```c

#include "config.h"
#ifdef CONSOLE_ENABLE
#   include "print.h"
#endif

#include QMK_KEYBOARD_H

#if __has_include("keymap.h")
#    include "keymap.h"
#endif

#ifdef OTHER_KEYMAP_C
#    include OTHER_KEYMAP_C
#endif // OTHER_KEYMAP_C

enum custom_keycodes {
    PY_PAR = SAFE_RANGE,
    PY_BRK,
    PY_CRL,
    EM_FYOU,
    NM_THZ,
    BT_GRB,
};

bool process_record_user(uint16_t keycode, keyrecord_t *record) {
    /*
    #ifdef CONSOLE_ENABLE
        const bool is_combo = record->event.type == COMBO_EVENT;
        uprintf("0x%04X,%u,%u,%u,%b,0x%02X,0x%02X,%u\n",
            keycode,
            is_combo ? 254 : record->event.key.row,
            is_combo ? 254 : record->event.key.col,
            get_highest_layer(layer_state),
            record->event.pressed,
            get_mods(),
            get_oneshot_mods(),
            record->tap.count
            );
    #endif
    */
    switch (keycode) {

        case PY_PAR:
            if (record->event.pressed) {
                SEND_STRING("()" SS_TAP(X_LEFT));}
            return false;
        case PY_BRK:
            if (record->event.pressed) {
                SEND_STRING("[]" SS_TAP(X_LEFT));}
            return false;
        case PY_CRL:
            if (record->event.pressed) {
                SEND_STRING("{}" SS_TAP(X_LEFT));}
            return false;
        case EM_FYOU:
            if (record->event.pressed) {
                send_unicode_string("???(-_-)???");}
            return false;
        case NM_THZ:
            if (record->event.pressed) {
                SEND_STRING(",000");}
            return false;
        case BT_GRB:
            if (record->event.pressed) {
                SEND_STRING("set prefix=(hd1,gpt2)/boot/grub;set root=(hd1,gpt2);insmod normal;normal");}
            return false;
    }
    return true;
};

enum {
    TD_Q_ESC,
};

// Tap Dance definitions
tap_dance_action_t tap_dance_actions[] = {
    // Tap once for Escape, twice for Caps Lock
    [TD_Q_ESC] = ACTION_TAP_DANCE_DOUBLE(KC_Q, KC_ESC),
};

// Ctrl-Alt-Del key combination
#define KC_CAD LALT(LCTL(KC_DEL))

const uint16_t PROGMEM keymaps[][MATRIX_ROWS][MATRIX_COLS] = {
    [0] = LAYOUT_split_3x5_2(
    TD(TD_Q_ESC),KC_W,KC_E,KC_R,KC_T,
    KC_Y,KC_U,KC_I,KC_O,KC_P,
    //
    KC_A,LALT_T(KC_S),LCTL_T(KC_D),LSFT_T(KC_F),KC_G,
    KC_H,RSFT_T(KC_J),RCTL_T(KC_K),LALT_T(KC_L),KC_SCLN,
    //
    LGUI_T(KC_Z),KC_X,KC_C,KC_V,KC_B,
    KC_N,KC_M,KC_COMM,KC_DOT,RGUI_T(KC_SLSH),
    //
    LT(1,KC_TAB),LT(2,KC_SPC),
    LT(3,KC_BSPC),LT(4,KC_ENT)),
    //
    [1] = LAYOUT_split_3x5_2(
    KC_GRV,KC_TILD,KC_EXLM,KC_AT,KC_HASH,
    KC_DLR,KC_PERC,KC_CIRC,KC_AMPR,KC_ASTR,
    //
    KC_UNDS,KC_LCBR,KC_LBRC,KC_LPRN,UC(0x2191),
    UC(0x2192),KC_RPRN,KC_RBRC,KC_RCBR,KC_DQUO,
    //
    UC(0x00F7),KC_EQL,KC_PMNS,KC_PLUS,UC(0x2190),
    UC(0x2193),KC_BSLS,KC_PIPE,KC_QUOT,UC(0x00B0),
    //
    KC_TRNS,KC_TRNS,
    KC_TRNS,KC_TRNS),
    [2] = LAYOUT_split_3x5_2(
    KC_F4,KC_F3,KC_F2,KC_LEFT,KC_RIGHT,
    KC_EQL,KC_7,KC_8,KC_9,KC_ASTR,
    //
    KC_F8,KC_PSCR,KC_CAPS,KC_ENT,KC_UP,
    KC_DOT,RSFT_T(KC_4),RCTL_T(KC_5),LALT_T(KC_6),KC_PMNS,
    //
    KC_F12,KC_F11,KC_F10,QK_BOOT,KC_DOWN,
    KC_0,KC_1,KC_2,KC_3,KC_PLUS,
    //
    KC_TRNS,KC_TRNS,
    KC_TRNS,KC_TRNS),
    [3] = LAYOUT_split_3x5_2(
    KC_LSFT,KC_HOME,KC_INS,KC_DEL,KC_END,
    KC_LEFT,KC_RGHT,XXXXXXX,UC_NEXT,KC_VOLU,
    //
    KC_LCTL,MS_WHLL,MS_WHLU,MS_WHLR,KC_PGUP,
    KC_UP,MS_LEFT,MS_UP,MS_RGHT,KC_VOLD,
    //
    KC_LALT,MS_BTN1,MS_WHLD,MS_BTN2,KC_PGDN,
    KC_DOWN,MS_BTN1,MS_DOWN,MS_BTN2,KC_MUTE,
    //
    KC_TRNS,KC_TRNS,
    KC_TRNS,KC_TRNS),
    [4] = LAYOUT_split_3x5_2(
    XXXXXXX,XXXXXXX,XXXXXXX,XXXXXXX,XXXXXXX,
    XXXXXXX,XXXXXXX,XXXXXXX,XXXXXXX,XXXXXXX,
    //
    XXXXXXX,PY_CRL,PY_BRK,PY_PAR,XXXXXXX,
    XXXXXXX,NM_THZ,EM_FYOU,KC_CAD,XXXXXXX,
    //
    XXXXXXX,XXXXXXX,XXXXXXX,XXXXXXX,XXXXXXX,
    XXXXXXX,XXXXXXX,XXXXXXX,XXXXXXX,BT_GRB,
    //
    XXXXXXX,XXXXXXX,
    XXXXXXX,XXXXXXX)};

bool caps_word_press_user(uint16_t keycode) {switch (keycode) {
        // Keep Caps Word active for letters
        case KC_A ... KC_Z:
            add_weak_mods(MOD_BIT(KC_LSFT)); // Apply Shift to make them uppercase
            return true;

        // Allow numbers,underscore,hyphen and backspace in Caps Word
        case KC_1 ... KC_0:
        case KC_UNDS:
        case KC_BSPC:
        case KC_PMNS:
            return true;

        // Otherwise, turn Caps Word off
        default:
            return false;}
};



const uint16_t PROGMEM llock_combo_left[] = {LT(1, KC_TAB), LT(2, KC_SPC), COMBO_END};
const uint16_t PROGMEM llock_combo_right[] = {LT(3, KC_BSPC), LT(4, KC_ENT), COMBO_END};

combo_t key_combos[] = {
    COMBO(llock_combo_left, QK_LLCK),
    COMBO(llock_combo_right, QK_LLCK),
};
```